Primo programma: MPLAB X e GPIO del PIC

Lezione 3 — 4MAK, SST-INF-01c — Firmware PIC: ripasso pre-stage

23/09/2026

1 Il microcontrollore PIC16F628A

Definizione

Il PIC16F628A appartiene alla famiglia **PIC16**, una famiglia di microcontrollori **a 8 bit**: la sua architettura interna lavora nativamente su canali a 8 fili per il trasporto dei dati.

- La lettera **F** indica la tecnologia **Flash**: la **memoria programma** può essere riscritta più volte, direttamente mentre il microcontrollore è montato nel circuito finale (**programmazione ICSP**, senza doverlo smontare).
- Il suffisso **A** indica la **revisione più recente** del modello: va sempre specificata, perché la versione senza `A` è più vecchia e leggermente diversa.

Nota

Prima della tecnologia Flash, programmare un microcontrollore richiedeva smontarlo dal circuito, montarlo su un **programmatore dedicato** e, per cancellarlo, esporlo per circa un quarto d'ora a raggi ultravioletti. Oggi la programmazione Flash avviene in pochi secondi, direttamente in circuito.

Definizione

Microchip produce centinaia di modelli di microcontrollori PIC a 8 bit, ciascuno con una dotazione diversa di memoria, **periferiche** e **piedini** disponibili (un po' come le versioni di una stessa auto, con optional diversi). Il PIC16F628A è un **modello volutamente semplice**, adatto a chi comincia: ha abbastanza risorse per essere utile, senza la complessità di modelli più grandi come il **PIC16F877A** (che verrà usato più avanti nel corso).

1.1 Il clock e la velocità di funzionamento

Definizione

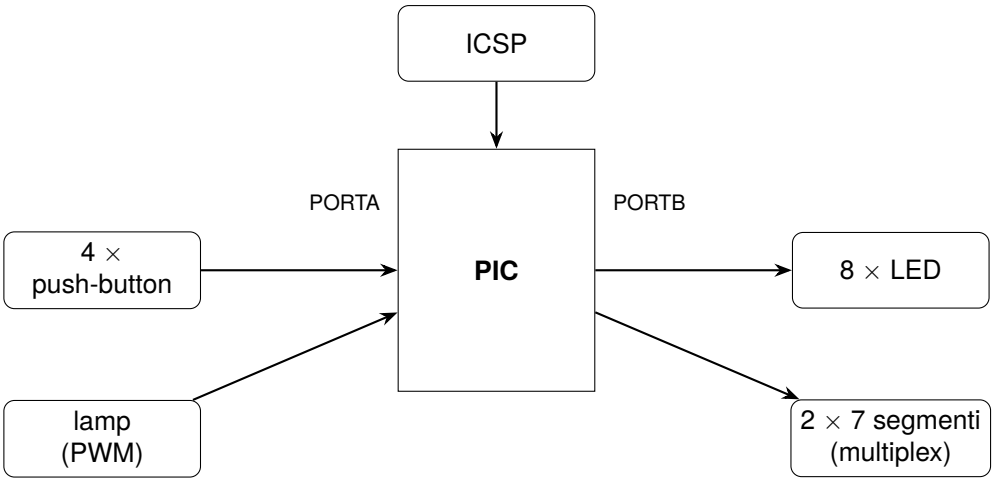
Ogni microcontrollore ha una **velocità massima** di **clock** dichiarata nel manuale del produttore (per il PIC16F628A, 20 MHz). Il clock è generato da un **oscillatore**, un dispositivo elettronico che produce un'**onda quadra**: ogni impulso fa avanzare di un passo l'esecuzione del programma.

Attenzione

Andare più veloci **non è sempre meglio**: un clock più veloce consuma più energia e scalda di più il circuito. Per applicazioni che non richiedono grande potenza di calcolo (es. un sistema che controlla l'umidità di una pianta, dove anche un calcolo di qualche secondo è ininfluente), conviene usare un **clock più lento** per risparmiare energia e prolungare l'**autonomia a batteria**.

2 La scheda MCLab1 e il simulatore PICSimLab

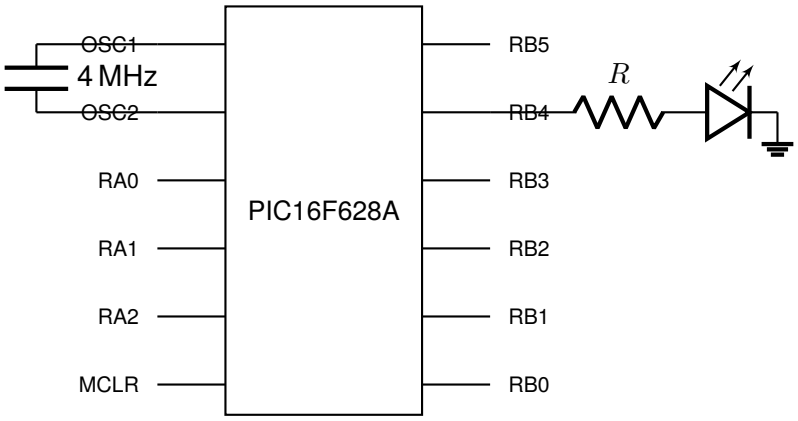
Per programmare senza ancora costruire hardware reale, si usa il **simulatore** **PICSimLab** con la **scheda virtuale** **MCLab1**: emula un circuito reale con PIC16F628A, pulsanti, **LED** e display a 7 segmenti.



Nota

In questo circuito, per costruzione, i piedini del gruppo **PORTA** sono collegati ai **pulsanti (input)** e alla lampada PWM, mentre i piedini del gruppo **PORTB** sono collegati a **LED** e display a 7 segmenti (output). Questa **non è una regola generale** del PIC: è lo **schema elettrico** del circuito specifico a decidere quali piedini sono **input** e quali **output** — un altro circuito potrebbe collegare LED anche sul gruppo A.

Lo **schema elettrico** completo della scheda MCLab1 mostra, oltre al microcontrollore, un **LED collegato a RB4** tramite resistore, un **quarzo** da 4 MHz collegato su **OSC1/OSC2**, e il **pulsante di reset** (collegato a MCLR):



3 I registri TRIS e PORT

Definizione

Ogni gruppo di piedini (A e B) è controllato da **due registri a 8 bit**, uno per bit di ciascun piedino:

- **TRIS** (`TRISA`, `TRISB`) decide se ciascun piedino è **input** o **output**: bit a `0` = **output** (la O di "Output" assomiglia allo 0), bit a `1` = **input** (la I di "Input" assomiglia all'1).
- **PORT** (`PORTA`, `PORTB`) determina, per i piedini configurati come **output**, **cosa** viene scritto verso l'esterno (acceso/spento); per i piedini configurati come **input**, permette di **leggere** lo stato imposto dall'esterno.

Esempio

Un'analogia utile: **TRIS** decide se un piedino è "**una bocca o un orecchio**" (**uscita o ingresso**); **PORT** determina cosa fa quella bocca (cosa dire, se è uscita) o cosa sente quell'orecchio (cosa viene letto, se è ingresso). Configurare la direzione (TRIS) e usarla (PORT) sono due passi distinti e vanno **sempre fatti entrambi**.

Attenzione

TRIS e PORT **non dicono la stessa cosa**: TRIS configura solo la **direzione** del segnale (dentro/fuori), non se il piedino è acceso o spento. Un piedino configurato come output ma non ancora scritto tramite PORT **non ha uno stato definito** e affidabile.

3.1 Pull-up e pull-down: leggere un pulsante in modo affidabile

Definizione

Un piedino configurato come input (pulsante) va collegato tramite una **resistenza** a un livello di riferimento, altrimenti il suo **stato non è definito** quando il pulsante non è premuto (piedino "**fluttuante**"). Esistono **due schemi opposti**:

- **Pull-up**: il piedino è collegato tramite resistenza a **VDD** (+5V). A riposo legge sempre

alto; premendo il pulsante (collegato a **massa**) legge **basso**.

- **Pull-down**: il piedino è collegato tramite resistenza a **massa** (0V). A riposo legge sempre **basso**; premendo il pulsante (collegato a **VDD**) legge **alto**.

Attenzione

Il pull-down sembra più intuitivo (si preme per ottenere "alto", cioè acceso), ma nella pratica si preferisce quasi sempre il **pull-up**. Il motivo è elettrico: ogni piedino configurato come input, anche se idealmente **non dovrebbe far passare corrente**, presenta in realtà una piccola **corrente di deriva** (o di dispersione) dovuta all'integrato. Con uno schema pull-up questa corrente di deriva scorre verso **VDD**, un nodo "rigido" che non risente di questa piccola perdita; con un pull-down la stessa corrente rischia di alterare leggermente il livello di massa letto dal piedino, con il rischio che il microcontrollore rilevi una **pressione del pulsante** che non è mai avvenuta. Per questo motivo il pull-up è considerato lo schema **più affidabile**, anche se meno intuitivo.

3.2 Il registro come griglia di 8 bit

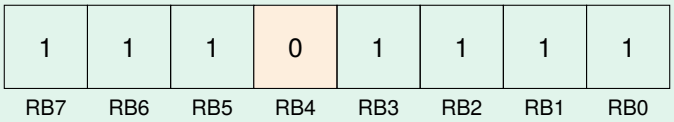
Nota

Ciascun registro a 8 bit si può immaginare come una **griglia di 8 caselle**, una per ciascun piedino del gruppo B: la casella più a sinistra è il **bit più significativo** (RB7), quella più a destra il **bit meno significativo** (RB0). Ogni casella vale 0 oppure 1, indipendentemente dalle altre.



Esempio

Configurazione di `TRISB` vista come **griglia**: si vuole `RB4` come **uscita** (per il LED) e tutti gli altri piedini come **ingressi**. Si scrive 0 nella casella RB4, 1 in tutte le altre:



Letta da sinistra a destra (RB7...RB0), la griglia dà `11101111` in binario: `TRISB = 0b11101111;` – coerente con il fatto che serve solo RB4 come uscita, come nell'esempio più semplice usato a lezione (`TRISB = 0x00;`, tutto il **gruppo B** come uscita, quando non interessa nessun ingresso su quel gruppo).

Esempio

Configurazione di `PORTB` per **accendere solo** il LED su RB4 (con RB4 già configurato come uscita in TRISB): si scrive 1 nella casella RB4, 0 in tutte le altre (i piedini configurati come

ingresso in TRISB vengono comunque **ignorati in scrittura**, ma per chiarezza si azzerano):

0	0	0	1	0	0	0	0
RB7	RB6	RB5	RB4	RB3	RB2	RB1	RB0

Letta da sinistra a destra: `00010000` in binario, cioè `PORTB = 0x10;` – lo stesso valore usato per accendere il LED nell'esempio di programma più avanti in questa lezione.

3.3 Scrivere il valore in binario, decimale o esadecimale

Esempio

Se si vuole configurare `RB3` come uscita, `RB4` come ingresso, e così via per tutti gli 8 bit, si può scrivere il valore del registro in **tre notazioni equivalenti**:

```
TRISB = 0b01100110;  // binario: si vede a colpo d'occhio ogni bit
TRISB = 102;          // decimale: compatto ma illeggibile bit a bit
TRISB = 0x66;         // esadecimale: compatto e leggibile a quartetti
```

Nota

La notazione **binaria** (prefisso `0b`) è la **più semplice da capire** perché mostra direttamente ogni bit, ma è lunga da scrivere e facile da sbagliare (contare 8 cifre a mano). La notazione **esadecimale** (prefisso `0x`) è quella **più usata dai programmatori esperti**: ogni cifra esadecimale rappresenta esattamente 4 bit (un **quartetto**), quindi un byte si scrive con solo 2 cifre. Ad esempio, `0b01100110` si divide nei quartetti `0110` e `0110`, entrambi pari a 6 in esadecimale: `0x66`.

4 Primo programma: accendere e far lampeggiare un LED

Nel **progetto MPLAB X** (categoria *Microchip Embedded*, tipo *Application Project*, microcontrollore PIC16F628A, compilatore **XC8**), il file principale si chiama per convenzione **`main.c`**.

Nota

MPLAB X è un **IDE** (Integrated Development Environment, ambiente di sviluppo integrato): un software che raggruppa **editor**, **compilatore** e strumenti di caricamento in un'**unica interfaccia**, per comodità. Concettualmente **non è indispensabile**: si potrebbe scrivere il codice in un editor di testo qualsiasi e lanciare il compilatore da riga di comando.

Attenzione

Nel menu per creare il nuovo file, MPLAB X propone anche un modello chiamato **`avr_main.c`**: è pensato per un'**altra famiglia** di microcontrollori (Microchip AVR, non PIC) e **non** va selezionato. Va scelto `main.c` semplice.

Esempio

Per accendere il LED collegato al piedino RB4:

```
#include <xc.h>

void main(void) {
    TRISB = 0x00;   // tutto il gruppo B come uscita
    PORTB = 0x10;   // solo il bit RB4 a 1: accende il LED su RB4
    while (1) {
        // ciclo infinito: non fa altro
    }
}
```

`0x10` in binario è `00010000`: **solo il quinto bit** (RB4, contando da RB0) è a 1, tutti gli altri a 0.

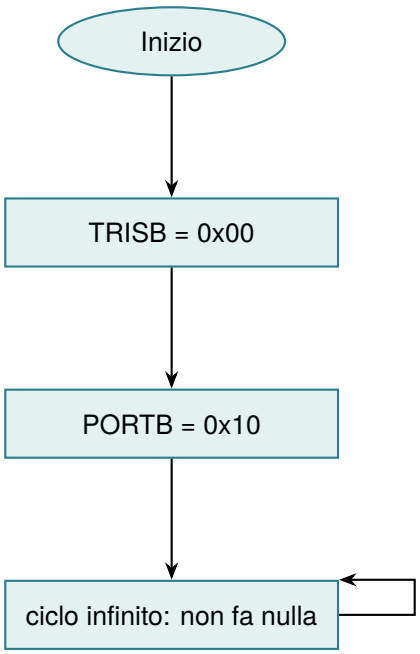


Figura 1: Diagramma di flusso: configurazione ed accensione fissa del LED, seguite da un ciclo infinito vuoto che blocca il programma senza fare altro.

4.1 Far lampeggiare il LED

Definizione

Per un'accensione fissa, l'istruzione **`PORTB = 0x10;`** va scritta **prima** del **`while`**, una volta sola. Per farlo **lampeggiare**, invece, accensione e spegnimento vanno spostate **dentro** il `while`, perché devono **ripetersi continuamente**:

```
while (1) {
    PORTB = 0x10;   // accende RB4
    PORTB = 0x00;   // spegne tutto
}
```

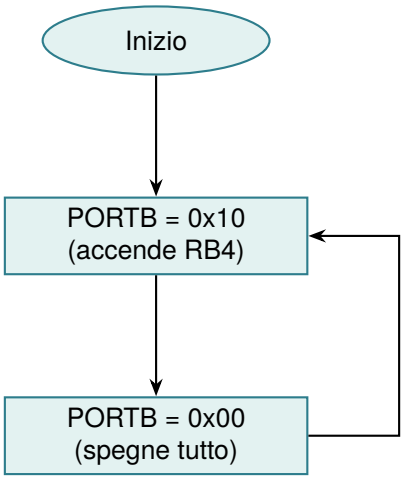


Figura 2: Diagramma di flusso del lampeggio senza ritardi: il ciclo `while(1)` ripete accensione e spegnimento senza sosta, troppo velocemente per essere percepiti.

Attenzione

Un microcontrollore è talmente veloce che, **senza attese esplicite**, l'accensione e lo spegnimento avvengono migliaia di volte al secondo: troppo rapido perché l'occhio umano (o perfino il circuito stesso) riesca a percepirlo. Ogni piedino ha piccole **capacità parassite** (**condensatori indesiderati**, generati dalla vicinanza fisica dei conduttori nel circuito integrato) che impiegano del **tempo a caricarsi**: se l'accensione dura meno di quel tempo di carica, il piedino non fa in tempo a raggiungere il livello alto prima che gli venga richiesto di tornare basso. Più il **clock è veloce**, più questo effetto peggiora.

4.2 I ritardi: `__delay_ms` e `__delay_us`

Definizione

Un modo rudimentale per rallentare è inserire un **ciclo `for`** che "non fa niente" per un certo numero di iterazioni, ma il numero di iterazioni necessario dipende dalla **velocità del microcontrollore**, ed è scomodo da calcolare a mano. La **libreria XC8** fornisce due funzioni pronte, **`__delay_ms(x)`** e **`__delay_us(x)`**, che attendono rispettivamente `x` millisecondi o microsecondi.
Per funzionare correttamente, queste funzioni devono sapere a quale frequenza gira il microcontrollore: va **dichiarato in cima al file** con una direttiva **`#define`**:

```
#define _XTAL_FREQ 20000000   // 20 MHz

void main(void) {
    TRISB = 0x00;
    while (1) {
        PORTB = 0x10;
        __delay_ms(6);
        PORTB = 0x00;
        __delay_ms(10);
    }
}
```

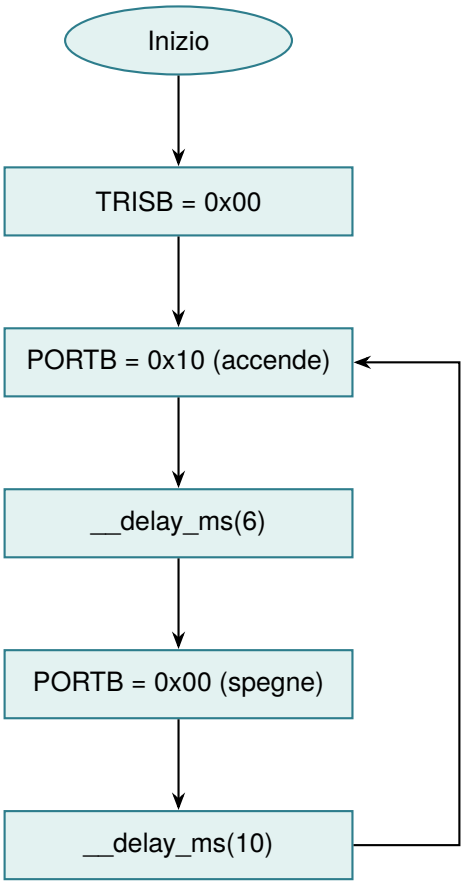


Figura 3: Diagramma di flusso del lampeggio con ritardi: dopo la configurazione iniziale, il ciclo `while(1)` alterna accensione/attesa e spegnimento/attesa, ripetendosi all'infinito.

Attenzione

Il parametro delle funzioni `__delay_ms`/**`__delay_us`** è un numero **intero**: **non si può scrivere** `__delay_ms(0,1)`. Per ottenere un ritardo di 0,1 millisecondi si usa **`__delay_us(100)`** (100 microsecondi equivalgono a 0,1 millisecondi) — è importante non confondere le unità di misura.

Esempio

Con **`__delay_ms(5)`** sia in accensione che in spegnimento, il **periodo totale** è $5 + 5 = 10$ millisecondi. La frequenza di lampeggio è $f = \frac{1}{P} = \frac{1}{10 \times 10^{-3}\ \text{s}} = 100$ Hz: 100 accensioni e spegnimenti al secondo.

5 Le direttive di configurazione (*configuration bits*)

Definizione

Prima del codice vero e proprio, un programma per PIC deve contenere alcune **direttive di sistema** che **configurano** il funzionamento di base del microcontrollore: tipo di oscillatore, uso o meno del **watchdog**, **protezione del codice**, eccetera. In MPLAB X si impostano

graficamente da *Window* → *Target Memory View* → *Configuration Bits*, e poi si copiano nel codice come una serie di **`#pragma config`**.

Nota

Per il circuito MCLab1 con **quarzo esterno**, la configurazione standard usata è: oscillatore **HS** (High Speed, a quarzo — coerente col quarzo collegato su OSC1/OSC2), **watchdog disattivato**, **power-up timer attivato**, **programmazione a basso voltaggio disattivata**.

Attenzione

Il **watchdog** è una **funzione di sicurezza** presente in quasi tutti i microcontrollori moderni: se il sistema si blocca, il watchdog lo riavvia automaticamente. Se non viene esplicitamente configurato (disattivato o gestito), il suo stato di default può essere **imprevedibile**: un watchdog attivo ma non gestito può causare **riavvii continui** del programma, con sintomi che sembrano un baco nel codice (es. un LED che lampeggia "da solo" senza che il codice lo preveda).

Definizione

Tra le opzioni di configurazione, **CP** (*Code Protection*) decide se il codice macchina caricato nel microcontrollore può essere **estratto** da chi acquista il prodotto finito. Se CP è disattivato, **chiunque smonti il circuito** può leggere (con qualche sforzo) il codice ed eventualmente **clonarlo**; se attivato, l'estrazione non è possibile. Per un prodotto commerciale, attivare la **protezione del codice** è quasi sempre la scelta corretta.

6 Compilare e caricare il programma

Definizione

Il pulsante ***Build Project*** in MPLAB X lancia il **compilatore XC8**, che traduce il codice C in **codice macchina** e produce un **file eseguibile** (`.hex` in formato testo, o `.production.hex`). Il pannello di output mostra un riepilogo della memoria occupata (memoria programma, memoria dati, configuration bits) e, se tutto è corretto, il messaggio **`BUILD SUCCESSFUL`**.

Nota

Il file compilato è in realtà una sequenza di cifre **esadecimali**: aprendolo con un editor di testo si vedono solo lettere fino alla `F` e numeri, **mai lettere oltre la F** — la conferma che si tratta di una **rappresentazione esadecimale** di sequenze di bit.

Una volta compilato, il **file `.hex`** va **caricato** nel simulatore **PICSimLab** tramite *File* → ***Load hex***, puntando al percorso generato dalla compilazione (tipicamente dentro `dist/default/production/` nella cartella del progetto).

Attenzione

Il progetto MPLAB X va salvato in una **cartella locale** (es. sul desktop), non direttamente in una cartella **sincronizzata con il cloud** (Google Drive, OneDrive): l'eseguibile che avvia il web server o il simulatore può **non partire correttamente** se il percorso è dentro una cartella sincronizzata. Il **backup su cloud** va fatto in un secondo momento, copiando la cartella già completa.

7 Misurare il segnale con l'oscilloscopio virtuale

Definizione

PICSimLab include un **oscilloscopio** virtuale: uno **strumento di misura** che mostra graficamente come varia nel tempo un segnale elettrico (in questo caso, la **tensione sul piedino RB4**). Permette di verificare quantitativamente la **frequenza di lampeggio** invece di stimarla "a occhio".

Esempio

Con un ritardo di 5 ms in accensione e 5 ms in spegnimento, l'oscilloscopio mostra un'**onda quadra** con periodo di 10 ms e frequenza di circa **100 Hz** — coerente con il calcolo teorico $f = 1/P$. Aumentando il clock del microcontrollore da 4 MHz a **20 MHz**, senza cambiare i ritardi nel codice, la **frequenza misurata resta la stessa**: i ritardi in millisecondi sono già compensati dal valore dichiarato in `_XTAL_FREQ`, indipendentemente dalla velocità reale del clock.

8 Cosa ricordare di questa lezione

- Il PIC16F628A è un microcontrollore a **8 bit Flash**, semplice, buon punto di partenza prima di modelli più complessi come il PIC16F877A.
- **TRIS** configura la **direzione** (input/output) di ciascun piedino bit per bit; **PORT** legge o scrive il **valore** sui piedini configurati come output.
- Un valore di registro può essere scritto in **binario, decimale o esadecimale**: l'esadecimale è il più usato perché compatto e leggibile a **quartetti di bit**.
- Per far lampeggiare un LED servono **ritardi** (`__delay_ms`/`__delay_us`) tra accensione e spegnimento, altrimenti la velocità del microcontrollore rende il cambiamento troppo rapido da percepire (e limitato dalle **capacità parassite** del circuito).
- Le **direttive di configurazione** (oscillatore, watchdog, protezione del codice) vanno sempre impostate esplicitamente in cima al programma, altrimenti il comportamento di default può essere **imprevedibile**.
- Il progetto MPLAB X va salvato in una **cartella locale**, non sincronizzata col cloud, per evitare problemi di avvio del simulatore.